

Software Engineering Fundamentals

Introduction to Software Engineering

Busting myths & preconceptions about SE



Dr. Petru Florin Mihancea

Presentation designed and written by human based on:

I. Sommerville - Software Engineering 8, Ch. I Introduction

R. Pressman - Software Engineering, Ch. I The Product

V20260223

I. What is Software Engineering?

The beginning ...

The “software crisis” started in the 60's

- Major programming projects were years late
- Software costs much higher than predicted
- Unreliable programs
- Difficult to maintain, understand and change the software
- etc.

Dr. Petru Florin Mihancea

The causes (I)

The major cause of the software crisis is that the **machines have become several orders of magnitude more powerful!** To put it quite bluntly: as long as there were no machines, programming was no problem at all; when we had a few weak computers, programming became a mild problem, and now **we have gigantic computers, programming has become an equally gigantic problem.**

Edsger Dijkstra

Complexity

Much larger and more complex software products than previously developed were hard to manage

Expectations

Hardware prices were going down, prices for software were increasing dramatically

Change

Need to maintain and evolve these programs but it was very hard

Dr. Petru Florin Mihancea

The causes (2)

Larger and more complex programs became possible, BUT ...

... the **informal** way of developing software became insufficient!

Dr. Petru Florin Mihancea

Software Engineering

... is an engineering discipline that is concerned with **all the aspects of software production** from the early stages of system specification to maintaining the system after it has gone into use

Firstly defined in 1968 at the NATO Software Engineering Conference

Why a different discipline is needed?

Dr. Petru Florin Mihancea

“Computer Science covers these issues”

Wrong! Mass-media myth

What is the difference between Software Engineering and Computer Science?

Computer science

Concerned with the fundamental theories and principles based on which computer and software systems are functioning

Software engineering

Concerned with the **practical problem of producing software products**

- It is not just concerned with the **technical processes** of software development but also with activities such as software project management and with the development of tools, methods and theories to support software production
- **Software engineers** apply CS theories but they use them selectively, try to discover solutions to problems even when there are no applicable theories and, look for solutions within given constraints e.g., financial restriction

Dr. Petru Florin Mihancea

“The only deliverable work product for a successful project is the working program”

Wrong! Developer myth

What is a software product?

A. Instructions (computer programs) that when executed provide desired function + data structures that enable programs to manipulate information

+

B. Documents that describe the operation and use of the programs (e.g. documentation)

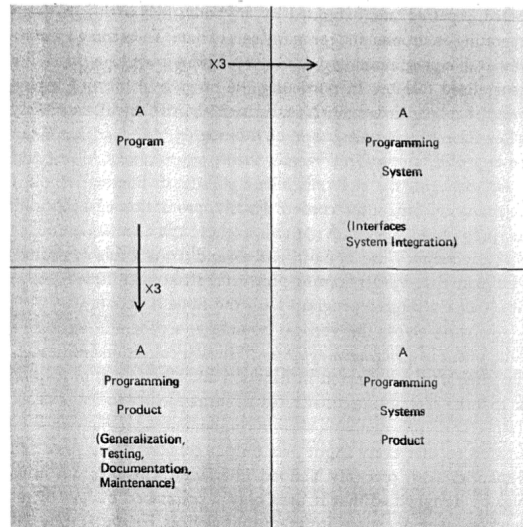
+

...

Real products are more than just code ...

Dr. Petru Florin Mihancea

Lab/Garage Programs vs Real Software Products



The Mythical Man-Month [Brooks, 1995]

Dr. Petru Florin Mihancea

“Once we write the program and get it work our work is done”

Wrong!
Developer myth

Which are the main development activities to apply in order to build a software product?

Fundamental software production activities

1. **Software specification**
defining the software to be produced
2. **Software development**
designing, programming/coding, testing (+debugging)
3. **Software validation**
testing to ensure that it is what the customer requires
4. **Software evolution/maintenance**
Software specification, development and, validation for modifications to adapt to customer/market requirements

A software development process is a set of activities and associated results that produce a software product

Dr. Petru Florin Mihancea

Some processes

Various ways in which software development activities can be “chained”

A. Waterfall

separate steps for specification, design, implementation, etc.
complete a phase and only after move to the next one

B. Iterative

interleaves fractions of specification, development, validation activities, etc.
fast development of an initial system and refine it with the customer

**Various pros/cons
- see later :)**

Dr. Petru Florin Mihancea

What fundamentals ?

Processes

what are the phases and how is the flow of a development process

Methods

how to accomplish different phases of a development process

Tools

what are the tools to support the involved activities

Other programming language mechanisms

used in professional software development (based on time)

Dr. Petru Florin Mihancea

Computer-Aided Software Engineering Tools

A wide range of different types of programs that are used to support software process activities

Upper CASE

front-end activities e.g. requirements and design



Lower CASE

back-end activities e.g. programming, debugging, testing



Dr. Petru Florin Mihancea

2. Distinguishing software product characteristics

Dr. Petru Florin Mihancea

A. What is a software product ?

Computer programs and their documentation

describing their operation and usage (e.g., design, user manuals)

Types of software products

Generic

developed by an organisation and sold on an open market; the organisation controls the specification; e.g. Word, Excel

Customised

developed especially for a particular customer according to their specification; e.g. air traffic control

Currently, more and more generic products are built which can also be adapted for specific customers e.g., Enterprise Resource Planning (ERP)

Couldn't a software product be developed / built like other things humans build?

Dr. Petru Florin Mihancea

Distinguishing characteristics of software

1. Software is **developed or engineered**, it is not manufactured in the classical sense

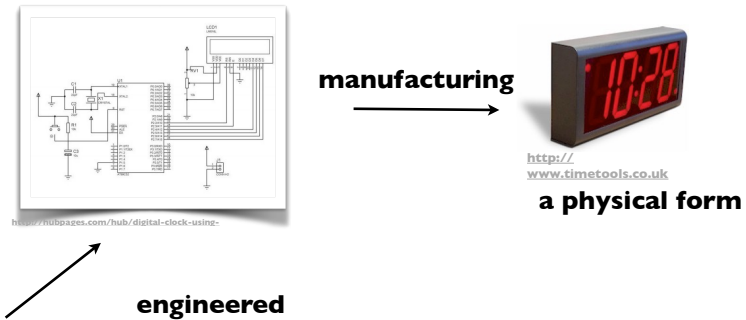
2. Software does not **"wear out"** in the classical sense

3. Although the industry is moving towards **"component-based assembly"**, most software continues to be "custom built"

Dr. Petru Florin Mihancea

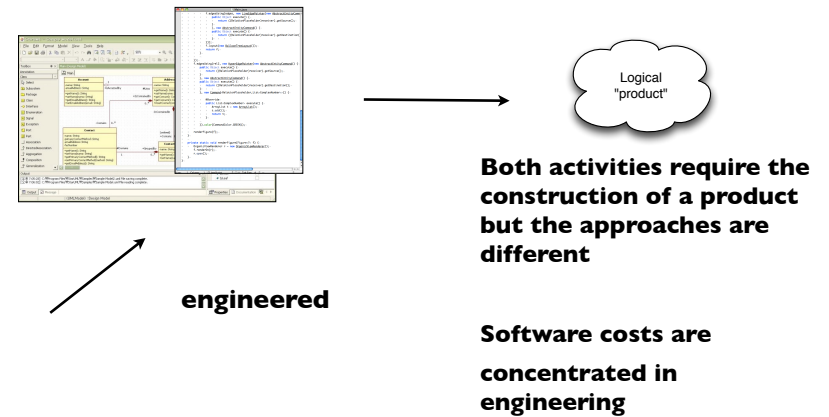
Software is **developed or engineered**, it is not manufactured in the classical sense (I)

for hardware



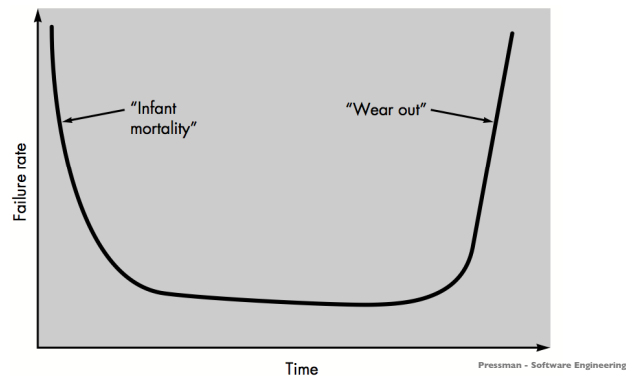
Software is **developed or engineered**, it is not manufactured in the classical sense (II)

for software

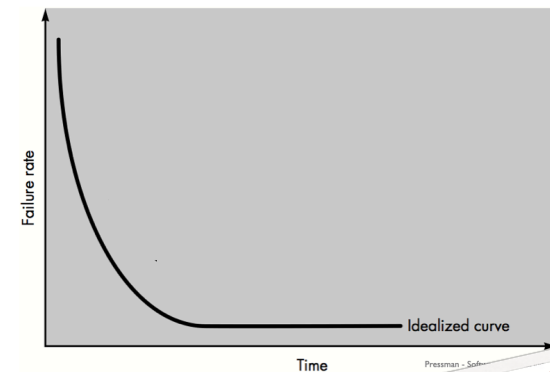


Software does not “wear out” in the classical sense (I)

for hardware



for software



But it deteriorates/“wears out” in a different sense

Software **must** evolve

Software systems must be **continually** adapted to new requirements else they become **progressively less satisfactory**

One of the Lehman's software evolution laws

Why ?

Dr. Petru Florin Mihancea

Software **aging**

Programs, like people, get old!

How ?

D. Parnas - Software aging

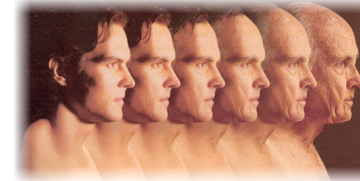


Changes are not necessarily performed by the original developers

Those that modify a system do not fully understand it and thus, may deteriorate the program (including its internal structure)

New **changes** cannot be accommodated in the original product design

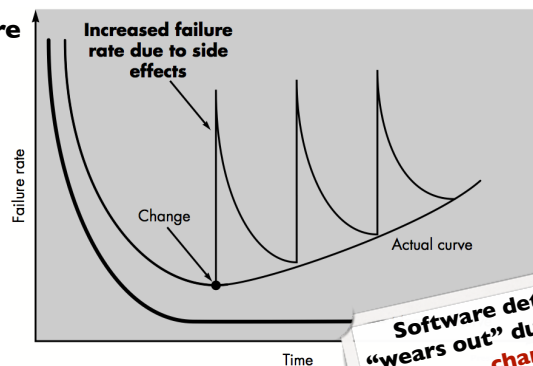
After several such modifications *nobody* understand the modified product :)



Dr. Petru Florin Mihancea

Software does not “wear out” in the classical sense (I)

for software



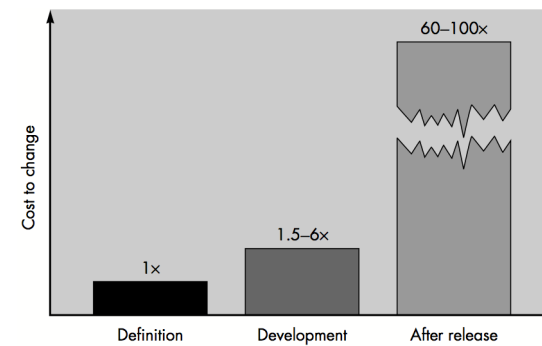
Software deteriorates/ “wears out” due to **repeated changes**

Software maintenance **considerably more difficult** than hardware maintenance

Dr. Petru Florin Mihancea

“Project requirements continually change, but change can be easily accommodated because software is flexible”

Wrong!
Customer myth

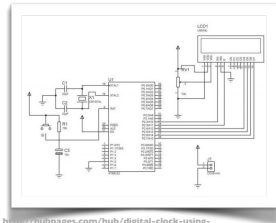


Pressman - Software Engineering

Dr. Petru Florin Mihancea

Software is still "custom built"

for hardware



select standard components,
having well defined functions,
interfaces, etc.



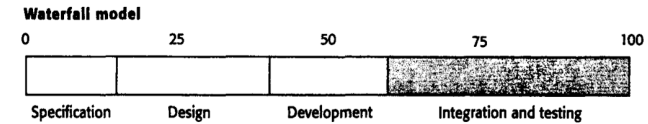
In the **software** world, this
component reuse is something that
has only begun to be achieved

We need more than algorithmic
libraries e.g., frameworks

Dr. Petru Florin Mihancea

B. What are the software **costs**? (I)

How is the software cost **distributed** over
different activities e.g., specification, design,
development, testing?



Coding is **NOT** the
main cost :)

And these are only **initial**
production costs!

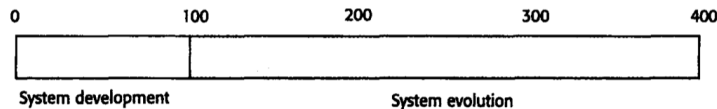
Dr. Petru Florin Mihancea

What are the software **costs**? (II)

"Once we write the program and get it work
our work is done"

Wrong!
Developer myth

Development and evolution costs for long-lifetime



Sommerville - Software Engineering

Always, design your
product for **change!**

Dr. Petru Florin Mihancea

C. What is **quality** for software?

Functional suitability e.g., correctness - it behaves/
functions as per software requirement specifications

Maintainability - software should be written in such a
way that it may evolve to meet the changing needs of
customers e.g., *modularity, testability*

Usability - it must be usable, without effort, by the type
of user for whom it is designed e.g., *user interface
aesthetics, learnability*

Efficiency - it should not make wasteful use of system
resources such as memory and processor e.g., *time
behaviour*

Portability, Reliability, Security, ...

The expected non-
functional quality factors
depend on the
application and not
possible to optimise it for
all these factors

Dr. Petru Florin Mihancea

3. Further **myths** and **preconceptions** about software development

“Software engineering will make us create voluminous and unnecessary documentation and will invariably slow us down”

Wrong!
Developer myth

Software engineering is **NOT** about documents, it is about creating **quality**

Better quality leads to reduce rework, reduce rework results in faster delivery times (and lower costs).

Dr. Petru Florin Mihancea

Dr. Petru Florin Mihancea

“Until I get the program “running” I have no way of assessing its quality”

Wrong!
Developer myth

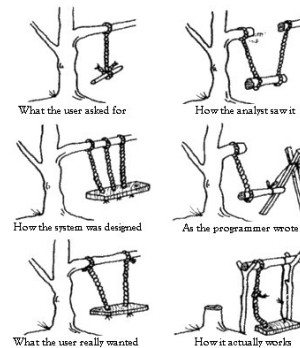
Software reviews can be applied from the inception of the project and found to be effective in finding certain classes of problems

Some analyses can be run while writing the code

Dr. Petru Florin Mihancea

“A general statement of objectives is sufficient to begin writing programs - we can fill details later”

Wrong!
Customer myth



A poor up-front definition is a major cause of failed projects

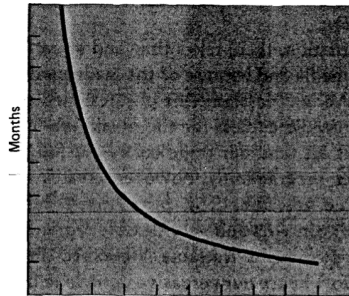
Communication with the customer is mandatory

Detailed (formal) description of domain, function, behaviour, performance, interfaces, validation criteria are essential

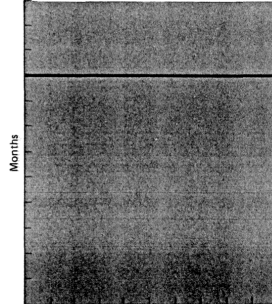
Dr. Petru Florin Mihancea

“If we get behind schedule, we can add more programmers and catch up”

Wrong! Manager myth



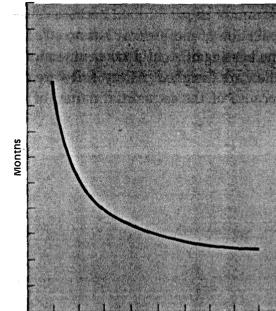
Completely independent subtasks



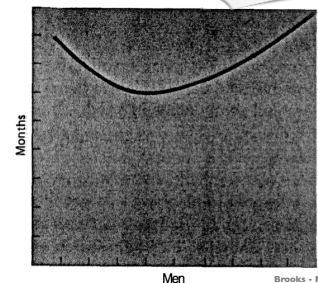
Non decomposable task

“If we get behind schedule, we can add more programmers and catch up”

Wrong! Manager myth



**Partially independent tasks
(extra time for communication)**



... and when too much communication is required :)

Brook's law - Adding manpower to a late software project makes it later

End of Chapter I

Live Quiz



**The exam has a different structure
(check the Exam section on CV)**